

Conflict-Architecture Arbitration Simultaneously Reduces Latency and Compute Energy in Real-Time Autonomous UAV Navigation

Vikhyat Chauhan and JoAnn M. Paul

Abstract—The Conflict Architecture (CA) is a multiple-instruction, single-data (MISD) organization that runs heterogeneous algorithms in parallel on one input and commits whichever candidate is ready when an external deadline forces selection. Prior work demonstrated the principle on maze-solving with time as the sole physical parameter. We (i) characterize the class of problems for which the CA is advantageous, (ii) instantiate the CA for autonomous aerial navigation in a Gazebo–ROS 2 testbed of procedurally generated corridor worlds under a stochastic, event-driven deadline model, and (iii) introduce algorithmic computation energy as an explicit arbitration metric. Across 1,000 runs per strategy, two higher-depth planners are infeasible as standalone strategies (deadline violations in 80.3% and 86.3% of runs); only a reflex baseline and the CA never miss a deadline. Between these two feasible strategies, the CA reduces mission time by 34.55% and algorithmic computation energy by 34.52% (both $p < 0.0001$), while no-fly-zone violations and total flight energy are statistically unchanged. The CA is therefore Pareto-improving over the only other deployable strategy, reducing latency and compute energy *simultaneously*—in contrast to the usual trade of performance against power.

Index Terms—MISD, conflict architecture, real-time systems, energy-aware computing, autonomous navigation, brain-inspired architecture.

I. INTRODUCTION

REAL-TIME autonomous systems face a structural dilemma: the best available action depends on how much time is available to compute it, and that budget is not known at design time. A single fixed pipeline must either commit to a fast, low-quality algorithm or risk a high-quality algorithm that may not finish before a response is required.

This tension is the engineering analogue of the speed–accuracy tradeoff in biological decision systems, where fast reflexive pathways and slower deliberative pathways operate in parallel and commitment is deferred until a physical event forces it.

The Conflict Architecture (CA) addresses this by organizing computation as multiple isolated algorithm processing elements (APEs) that operate on a shared input and produce conflicting candidate outputs; an external physical event selects among completed candidates rather than forcing early logical convergence [1]. This places the CA in Flynn’s MISD category [2], historically dismissed as either incoherent or

useful only for fault detection. The CA reframes conflicting answers as the design rather than a defect.

Prior work established the CA as a proof of concept on maze-solving under unpredictable pursuit, using elapsed time as the only selection parameter [1]. That work left open which classes of real-world problems benefit from the approach, and whether the advantage survives under multiple simultaneous physical constraints—particularly energy.

This letter answers both for autonomous UAV navigation, where timing, energy, and spatial safety must be satisfied at once and no single strategy is optimal across operating conditions. Our contributions are:

- 1) a characterization of the problem class for which the CA is advantageous (Sec. II);
- 2) a working CA instantiation for UAV navigation, evaluated in a Gazebo–ROS 2 testbed across 1,000 runs per strategy (Secs. III, IV);
- 3) the introduction of *algorithmic computation energy* as an explicit arbitration metric (Sec. IV); and
- 4) the first evidence that CA arbitration is Pareto-improving across physically meaningful metrics—reducing mission time and compute energy together while holding safety and flight energy constant (Sec. V).

II. WHEN IS THE CA ADVANTAGEOUS?

The CA is not universally beneficial; it is advantageous for problems exhibiting four properties together, none individually sufficient:

- 1) **Critical decisions under unpredictable deadlines.** The environment, not the algorithm, dictates when an action must be produced, and the available interval varies from near-instantaneous to extended.
- 2) **Multiple structurally distinct strategies.** Valid strategies differ by orders of magnitude in computation, memory, and energy profile—e.g., reflexes on raw sensor data, local planners on immediate geometry, and global planners on stable maps—and no single one dominates across conditions.
- 3) **Feasibility set by physical constraints, not optimality.** A globally optimal plan that arrives after a collision deadline or exhausts energy is infeasible; a timely sub-optimal maneuver succeeds. Feasibility is effectively binary; optimality is secondary.
- 4) **Continuous interaction with a dynamic, partially observable environment.** Assumptions made during

The authors are with the Bradley Department of Electrical and Computer Engineering, Virginia Tech, Blacksburg, VA 24061 USA (e-mail: jmpaul@vt.edu).

This work was supported by the National Science Foundation under Grant 2204780.

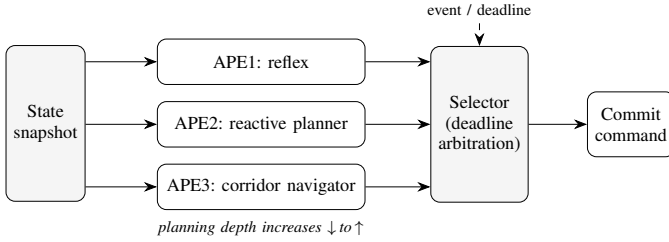


Fig. 1. CA Navigation Engine. Three APEs labeled by their navigation algorithm run in parallel on one state snapshot; the Selector commits the deepest completed proposal before each event deadline and falls back to the APE1 reflex otherwise. (Redrawn for this work; cf. the generic CA in [1].)

planning can be invalidated before execution, so maintaining multiple candidates avoids costly replanning.

Offline optimization, perception-only tasks, and scripted control with deterministic timing lack these properties and do not benefit from the CA. Autonomous aerial navigation exhibits all four and is our testbed.

III. CONFLICT ARCHITECTURE FOR UAV NAVIGATION

The CA-based Navigation Engine runs three APEs in parallel on a single time-stamped state snapshot and commits, before each externally imposed deadline, whichever proposal is both complete and highest in planning depth (Fig. 1):

- **APE1 — reflex (hard fail-safe).** A fixed-sidestep maneuver with minimal computation that always completes within the tightest inter-event interval, guaranteeing a response.
- **APE2 — reactive planner (moderate depth).** Evaluates local corridor geometry with side-bias memory before committing.
- **APE3 — corridor navigator (high depth).** Scores candidate headings across a sweep range using forward clearance, arc-safety checks, and no-fly-zone (NFZ) proximity penalties.

A Selector commits the best completed proposal at each deadline; if neither APE2 nor APE3 has finished, it falls back to APE1. Because APE1 is always available, no deadline is ever missed regardless of inter-event interval. The generic CA formulation and its maze instantiation appear in [1] and are not reproduced here.

IV. METHODOLOGY

A. Testbed and protocol

All strategies are evaluated in Gazebo–ROS 2 on procedurally generated Manhattan-grid corridor worlds, where corridor walls act as NFZ constraints and dynamic obstacles appear during flight. For each run, all four strategies (APE1, APE2, APE3, CA) are evaluated on identical world geometry and event sequence via a fixed seed, so geometry and hazard timing are constant across strategies; across 10 replications seeds vary to broaden coverage. We report 1,000 runs per strategy. A passive measurement pipeline records pose, velocity, and orientation as time-stamped traces without perturbing timing,

TABLE I
PER-APE DOMINANT ARITHMETIC AND NATIVE LATENCY (ORIN NX, 2.0 GHz).

	APE1	APE2	APE3
Core arithmetic cycles (approx.)	~1,046	~2,686	~4,070
Native latency @ 2.0 GHz	523 μ s	1,343 μ s	2,035 μ s

and the four strategies execute in isolation so that one strategy’s computation cannot affect another’s timing. Only runs in which all strategies reach the target within a timeout are aggregated; all reported metric spreads remain within 5% of their means across replications.

B. Event-driven deadlines

Hazard events arrive on a bounded log-uniform inter-arrival distribution over $[0.60, 3.50]$ s, frequently short enough that high-depth planners cannot complete in time. Each event imposes a deadline by which a navigation command must be committed; a violation means no response was produced and, in deployment, corresponds to a collision.

C. Energy models

Physical flight energy is estimated post-hoc from pose traces using a distance-based energy-per-meter (EPM) model,

$$E_{\text{flight}} = \text{EPM} \sum_i d_i, \quad (1)$$

where d_i are horizontal displacement increments (≥ 0.01 m) and $\text{EPM} = 208.9$ J/m for a DJI FlyCart 30-class platform. This proxy captures the dominant transit cost and isolates trajectory-length differences between strategies.

Algorithmic computation energy integrates an active-power model over each APE’s invocation interval,

$$P = P_{\text{idle}} + (\text{TDP} - P_{\text{idle}}) U_{\text{eff}}, \quad (2)$$

with coefficients from an NVIDIA Jetson Orin NX target (8-core Cortex-A78AE @ 2.0 GHz, $\text{TDP} = 25$ W, $P_{\text{idle}} \approx 2.5$ W). Per-APE native latencies derived from Cortex-A78 cycle counts are 523 μ s, 1,343 μ s, and 2,035 μ s for APE1–3 (Table I); a $1000\times$ scaling maps core compute to an end-to-end embedded budget that preserves the APE cost ordering. The core finding is invariant to the absolute budget magnitudes: it requires only that $\text{APE1} < \text{APE2} < \text{APE3}$ and that APE1’s budget falls below the minimum deadline while APE3’s stays within the maximum.

V. RESULTS

A. Feasibility

The decisive finding is that the higher-depth planners are infeasible alone: APE2 violates deadlines in 80.3% of runs (mean violation rate 28.02%) and APE3 in 86.3% of runs (40.39%). Their nominally faster elapsed times (Table II) are artifacts of these violations and are not comparable. Only APE1 and the CA achieve zero deadline violations across all 1,000 runs—APE1 by construction, the CA by always retaining APE1 as a fallback. Performance comparison is therefore restricted to these two feasible strategies.

TABLE II
STRATEGY OVERVIEW AND FEASIBILITY (1,000 RUNS).

Strategy	Mean (s)	SD (s)	Median (s)	Violation rate
APE1	59.56	27.66	68.82	0.00%
APE2	34.70	16.86	29.80	28.02%
APE3	33.89	18.88	28.62	40.39%
CA	38.98	17.78	35.39	0.00%

Elapsed times for APE2/APE3 are uninterpretable due to violations; reported for algorithm comparison only.

TABLE III
APE1 VS. CA ACROSS 1,000 RUNS (ONLY FEASIBLE STRATEGIES).

Metric	APE1 (mean $\pm$ SD)	CA (mean $\pm$ SD)	Δ
Elapsed time (s)	59.56 $\pm$ 27.66	38.98 $\pm$ 17.78	−34.55%*
Compute energy (J)	148.92 $\pm$ 69.17	97.51 $\pm$ 44.49	−34.52%*
NFZ violations	0.576 $\pm$ 0.863	0.563 $\pm$ 0.919	−2.26%
Flight energy (J)	38,432 $\pm$ 5,255	38,922 $\pm$ 6,463	+1.28%
Avg. power (W)	815 $\pm$ 408	1,086 $\pm$ 256	+33.22%*

* $p < 0.0001$; remaining differences not significant ($p > 0.05$). Both feasible strategies: 0.00% deadline-violation rate.

B. Dual improvement (APE1 vs. CA)

Table III gives the head-to-head. The CA reduces mission elapsed time by 34.55% (59.56 $\rightarrow$ 38.98 s) and algorithmic computation energy by 34.52% (148.92 $\rightarrow$ 97.51 J), both at $p < 0.0001$ and consistent across all 10 replications. Median shifts (68.82 $\rightarrow$ 35.39 s) confirm the gain is not outlier-driven. This is architecturally significant: speed is conventionally bought with more computation, yet the CA reduces both at once, because it commits to the cheapest pathway sufficient for the current deadline rather than running a uniform reflex regardless of available time. The gain comes entirely from preferring deeper planners when they complete in time, with no increase in deadline violations.

C. Safety and energy held constant

NFZ violations are statistically indistinguishable (0.576 vs. 0.563 per run, $p > 0.05$; zero median for both), so the CA neither degrades nor improves safety. Total physical flight energy is neutral (+1.28%, $p > 0.05$): both strategies cover comparable distances. Average power rises 33.22% (815 $\rightarrow$ 1086 W, $p < 0.0001$)—the CA sustains higher instantaneous power over a shorter mission at equal total energy, a signature of more decisive navigation rather than inefficiency.

VI. CONCLUSION

Instantiating the CA for UAV navigation and adding compute energy as an arbitration metric, we show that conflict-driven, deadline-forced selection is Pareto-improving over the only other deployable strategy: it cuts mission latency and computation energy together while leaving safety and flight energy unchanged. The result generalizes the CA beyond its maze proof of concept to a multi-constraint, real-time domain, and indicates that MISD-style structural conflict is a useful organizing principle for energy-aware autonomous systems. Hardware deployment, finer-grained and neuromorphic APE families, and runtime-adaptive APE composition that suspends expensive planners under low energy are natural next steps.

REFERENCES

- [1] J. M. Paul and I. Bettendorf, “Paradox, conflict, and structural intelligence,” *Computer*, vol. 58, no. 2, pp. 35–41, Feb. 2025.
- [2] M. J. Flynn, “Very high-speed computing systems,” *Proc. IEEE*, vol. 54, no. 12, pp. 1901–1909, Dec. 1966.